

A Project Report on

Design and FPGA Implementation of a Significance-Driven Hybrid Approximate 8×8 Multiplier for Error-Tolerant Image Processing

Submitted in partial fulfilment of the requirements for the award of
the degree of

Bachelor of Technology

in

Electronics and Communication Engineering

by

Student Name 1 Roll No. XXXXXXXXX

Student Name 2 Roll No. XXXXXXXXX

Student Name 3 Roll No. XXXXXXXXX

Under the guidance of

Dr./Prof. Guide Name

Designation, Department of ECE

Department of Electronics and Communication Engineering

College Name

(Affiliated to University Name)

City – PIN, State, India

Month Year

Certificate

This is to certify that the project report entitled “**Design and FPGA Implementation of a Significance-Driven Hybrid Approximate 8×8 Multiplier for Error-Tolerant Image Processing**” submitted by **Student Name 1 (Roll No.)**, **Student Name 2 (Roll No.)** and **Student Name 3 (Roll No.)** in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering of **University Name** is a record of bona fide work carried out by them under my supervision during the academic year **20XX–20XX**.

Guide Name

Project Guide

Department of ECE

HOD Name

Head of the Department

Department of ECE

External Examiner

Date: _____

Declaration

We hereby declare that the project report entitled “**Design and FPGA Implementation of a Significance-Driven Hybrid Approximate 8×8 Multiplier for Error-Tolerant Image Processing**” is a record of original work carried out by us under the guidance of **Guide Name**, and has not been submitted elsewhere for the award of any degree or diploma. Wherever the work of others has been used, it has been duly acknowledged and referenced. Parts of the Verilog code and of this report were drafted with the assistance of an AI tool; we have reviewed, executed and verified all code and results, and we take full responsibility for the content.

Place: **City**

Date: **DD/MM/YYYY**

Student Name 1

Student Name 2

Student Name 3

Acknowledgement

We express our sincere gratitude to our project guide, **Guide Name**, for continuous guidance, encouragement and valuable suggestions throughout this work. We thank **HOD Name**, Head of the Department of Electronics and Communication Engineering, and **Principal Name**, Principal, **College Name**, for providing the facilities required for the project. We also thank the faculty and laboratory staff of the department, and our families and friends for their support.

Student Name 1

Student Name 2

Student Name 3

Abstract

Multiplication is the most resource-hungry arithmetic operation in digital signal processing, image processing and machine-learning hardware. Many of these applications are *error tolerant*: small numerical errors in individual products do not produce visible or meaningful changes in the final output. Approximate computing exploits this tolerance by deliberately simplifying arithmetic circuits to save hardware.

This project designs, verifies and evaluates a family of six 8×8 unsigned multipliers written in Verilog HDL for FPGA implementation. The multipliers are built recursively from 2×2 blocks. Approximation is applied selectively to the partial products of lowest significance, using either the under-designed 2×2 multiplier of Kulkarni *et al.* or complete removal (truncation) of the least-significant 4×4 partial product. The proposed hybrid design (HYB) combines both techniques.

All designs were verified exhaustively over the 65,536 possible input pairs with a self-checking testbench in Icarus Verilog, and cross-checked against an independent Python model. Synthesis with Yosys for the Xilinx 7-series (Artix-7) fabric of the Basys-3 board shows that HYB needs 80 LUTs instead of 112 for the exact design, a 28.6% reduction, with a normalised mean error distance of 0.24%. In three image-processing applications (image multiplication, Gaussian smoothing and image blending) computed from the RTL outputs, HYB achieves a PSNR of 46.7–50.1 dB and an SSIM of at least 0.996 compared with exact multiplication. The study also shows that, at the LUT granularity of FPGAs, removing the lowest partial product gives a better accuracy–area trade-off than approximating higher partial products.

Keywords: approximate computing, approximate multiplier, FPGA, Verilog HDL, Artix-7, Basys-3, image processing, PSNR, SSIM.

Contents

Certificate	ii
Declaration	iii
Acknowledgement	iv
Abstract	v
List of Figures	ix
List of Tables	x
List of Abbreviations	xi
1 Introduction	1
1.1 Background	1
1.2 Approximate Computing	1
1.3 Problem Statement	1
1.4 Objectives	2
1.5 Methodology Overview	2
1.6 Organisation of the Report	2
2 Literature Survey	4
2.1 Approximate Computing	4
2.2 Approximate Arithmetic Circuits	4
2.3 Under-Designed Multiplier	4
2.4 Architectural Exploration of Approximate Multipliers	4
2.5 FPGA-Specific Approximate Multipliers	5
2.6 Summary and Research Gap	5
3 Theoretical Background	7
3.1 Binary Multiplication	7
3.2 Recursive (Divide-and-Conquer) Multiplication	7
3.3 Exact and Approximate 2×2 Multipliers	7
3.4 Partial-Product Truncation	8
3.5 FPGA Architecture	8
3.5.1 Look-up tables and carry chains	8
3.5.2 Basys-3 board	9

3.6	Error Metrics	9
3.7	Image Quality Metrics	9
4	Proposed Design	10
4.1	Design Space	10
4.2	Properties of the Design Family	11
4.3	Verilog Implementation	11
4.3.1	2×2 blocks	11
4.3.2	4×4 block	12
4.3.3	8×8 multiplier	12
4.3.4	Registered wrapper	14
4.4	Basys-3 Demonstration System	14
5	Implementation and Verification Methodology	15
5.1	Tools	15
5.2	Exhaustive Functional Verification	15
5.3	Waveform Simulation	15
5.4	Synthesis Methodology	16
5.5	Image-Processing Applications	16
6	Results and Discussion	17
6.1	Functional Verification	17
6.2	Simulation Waveforms	17
6.3	Error Characteristics	18
6.4	FPGA Resource Utilisation	19
6.4.1	Discussion	19
6.5	Accuracy–Area Trade-off	20
6.6	Image-Processing Results	21
6.7	Basys-3 Demonstration Design	22
6.8	Limitations	22
7	Conclusion and Future Scope	23
7.1	Conclusion	23
7.2	Future Scope	23
	Bibliography	24
A	Verilog RTL Source Code	26
B	Testbenches	27
C	FPGA Implementation Files	30

D Synthesis and Analysis Scripts	35
---	-----------

List of Figures

1.1	Design and evaluation flow used in this project.	3
4.1	Architecture of the proposed hybrid (HYB) 8×8 multiplier. Each approximate 4×4 block is built from four under-designed 2×2 blocks. .	10
6.1	Simulation waveform of the six registered multipliers (Icarus Verilog, 100 MHz clock).	17
6.2	Histograms of non-zero error distances (logarithmic count axis). . . .	18
6.3	Relative error over the operand space (A vertical, B horizontal; colour scale clipped at 25%).	19
6.4	LUT count versus NMED (EXACT plotted at 10^{-4} for the logarithmic axis).	20
6.5	Output images of the three kernels for every multiplier (PSNR / SSIM against the exact output).	21

List of Tables

2.1	Summary of related work	6
3.1	Truth table of the exact and approximate 2×2 multipliers (products in decimal)	8
4.1	Multiplier configurations (MODE parameter of <code>mul8x8</code>)	10
4.2	Basys-3 user interface	14
6.1	Products observed in the waveform simulation	18
6.2	Error metrics over all 65,536 input pairs	18
6.3	Synthesis results for the Xilinx 7-series fabric (Yosys, DSP inference disabled)	19
6.4	Image quality compared with exact multiplication: PSNR (dB) / SSIM	21

List of Abbreviations

ASIC	Application-Specific Integrated Circuit
CARRY4	Xilinx 7-series 4-bit carry-chain primitive
CLB	Configurable Logic Block
DSP	Digital Signal Processing
ED	Error Distance
ER	Error Rate
FPGA	Field-Programmable Gate Array
HDL	Hardware Description Language
HYB	Proposed hybrid approximate multiplier
LUT	Look-Up Table
MED	Mean Error Distance
MRED	Mean Relative Error Distance
NMED	Normalised Mean Error Distance
PSNR	Peak Signal-to-Noise Ratio
RTL	Register-Transfer Level
SSIM	Structural Similarity Index Measure
UDM	Under-Designed Multiplier
WCE	Worst-Case Error
XDC	Xilinx Design Constraints

Chapter 1

Introduction

1.1 Background

Digital systems for image processing, audio processing, communications and machine learning perform very large numbers of multiplications. A single 3×3 convolution on a 512×512 image needs more than two million multiply operations, and a small neural network needs millions more. In hardware, the multiplier is usually the largest and most power-hungry arithmetic unit, so its cost dominates the cost of the whole datapath.

Field-Programmable Gate Arrays (FPGAs) are widely used to accelerate such workloads because they can implement many operations in parallel and can be reprogrammed. On an FPGA, logic is built from small look-up tables (LUTs), dedicated carry chains, and a limited number of hard DSP blocks. Low-cost FPGAs used in education and embedded products, such as the Artix-7 XC7A35T on the Digilent Basys-3 board, have only 90 DSP slices [1]. Once these are used up, every additional multiplier must be built from LUTs, and LUT-efficient multiplier design becomes important.

1.2 Approximate Computing

Many applications do not need exact results. The human eye cannot see a one-level change in a pixel value; a classifier usually outputs the same label even if its internal products carry small errors; and sensor data already contain noise. *Approximate computing* exploits this error tolerance by simplifying hardware so that it produces slightly inexact results in exchange for lower area, delay or energy [2, 3]. Arithmetic circuits, especially adders and multipliers, are the most studied targets [4].

1.3 Problem Statement

Most approximate multipliers in the literature were designed and evaluated for ASICs, where savings are counted in gates or transistors. On an FPGA, a design that removes a few gates may not remove any LUT, because a 6-input LUT implements any function of up to six inputs at the same cost. The problem addressed in this project is therefore:

Where, within a recursive 8×8 multiplier, should approximation be applied to obtain the best trade-off between LUT usage and output quality on a low-cost FPGA, and how do such multipliers perform in real image-processing tasks?

1.4 Objectives

1. Design an exact recursive 8×8 unsigned multiplier in Verilog HDL from 2×2 building blocks.
2. Design approximate variants that apply approximation at different significance levels, using the under-designed 2×2 multiplier and partial-product truncation, and propose a hybrid design.
3. Verify every design exhaustively with a self-checking testbench and produce simulation waveforms.
4. Characterise the error of every design using standard metrics (ER, MED, NMED, MRED, WCE).
5. Synthesise every design for the Xilinx 7-series (Artix-7) fabric and compare LUT, carry-chain and logic-depth figures.
6. Evaluate the multipliers in image multiplication, Gaussian smoothing and image blending using PSNR and SSIM.
7. Build a Basys-3 demonstration that lets a user select the multiplier and see the product and error on the board.

1.5 Methodology Overview

Figure 1.1 summarises the design flow. The RTL is written in Verilog, simulated with Icarus Verilog, synthesised with Yosys for the Xilinx 7-series fabric, and analysed with Python. All steps are scripted, so every number in this report can be regenerated with one command (`run_all.sh`).

1.6 Organisation of the Report

Chapter 2 reviews the literature on approximate computing and approximate multipliers. Chapter 3 presents the theory of binary multiplication, recursive decomposition, FPGA architecture and quality metrics. Chapter 4 describes the proposed multiplier

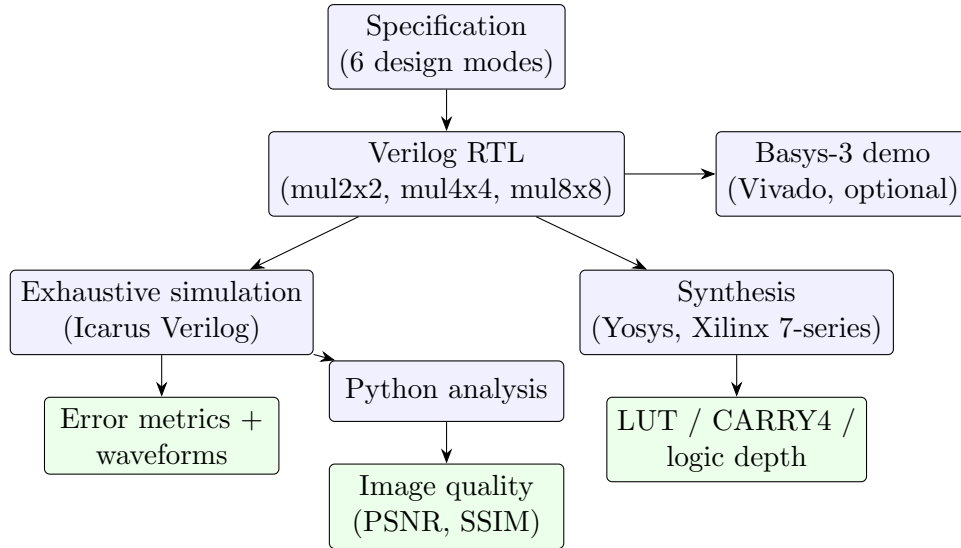


Figure 1.1: Design and evaluation flow used in this project.

family and its Verilog implementation. Chapter 5 explains the verification, synthesis and application methodology. Chapter 6 presents and discusses the results. Chapter 7 concludes the report and outlines future work. The appendices contain the complete source code and instructions to reproduce the results.

Chapter 2

Literature Survey

2.1 Approximate Computing

Han and Orshansky [2] described approximate computing as an emerging paradigm for energy-efficient design, in which the requirement for exact computation is relaxed for applications that can tolerate errors. Mittal’s survey [3] classifies approximation techniques at the circuit, architecture and software levels, and identifies image processing, multimedia and machine learning as the main beneficiaries. At the circuit level, approximate adders and multipliers are the most common building blocks.

2.2 Approximate Arithmetic Circuits

Jiang *et al.* [4] survey and characterise a large number of approximate adders, multipliers and dividers. They define the error metrics used in this report (error rate, mean error distance, normalised and relative error distances) and show that the best design depends strongly on the application and on the implementation technology. Approximate multipliers are generally built by (i) approximating the partial-product generation, (ii) approximating the partial-product accumulation, or (iii) truncating low-order bits or partial products.

2.3 Under-Designed Multiplier

Kulkarni, Gupta and Ercegovac [5] proposed an under-designed 2×2 multiplier block that is exact for 15 of its 16 input combinations; for 3×3 it outputs 7 instead of 9. Because the output then fits in three bits, the block is smaller, and larger multipliers built from it recursively inherit the saving. The resulting error is always an under-estimate. This block is the basis of three of the designs in this project.

2.4 Architectural Exploration of Approximate Multipliers

Rehman *et al.* [6] explored the architectural space of approximate multipliers built from approximate elementary blocks and approximate adders, showing that the place-

ment of approximate blocks has a large effect on the accuracy–cost trade-off. This project applies a similar placement idea, but targets FPGA LUT cost and adds partial-product truncation as a second technique.

2.5 FPGA-Specific Approximate Multipliers

Ullah *et al.* [7] showed that multipliers designed for ASICs do not necessarily give the same savings on FPGAs, and proposed accurate and approximate multipliers that exploit the LUT and carry-chain structure of Xilinx FPGAs. Vakili *et al.* proposed DyRecMul [8], which uses dynamically reconfigurable LUT primitives; they report LUT reductions of 64% for signed 8-bit multiplication compared with the standard Xilinx multiplier core, with an average accuracy loss below 0.29% on deep-learning benchmarks. Kida and Sato [9] showed that a carefully hand-structured exact 4-bit multiplier on 7-series FPGAs needs only 11 LUTs and two CARRY4 blocks, and that Vivado does not infer this structure from a behavioural $a*b$.

2.6 Summary and Research Gap

Table 2.1 summarises the reviewed work. Existing FPGA-oriented approximate multipliers achieve large savings but often rely on device-specific primitives, dynamic reconfiguration or hand-placed LUT initialisation, which makes them hard to reproduce in a student project. There is room for a simple, fully portable study that uses only standard Verilog, open-source tools and a low-cost board, and that answers where approximation should be placed within a recursive multiplier to minimise LUT usage for a given error.

Table 2.1: Summary of related work

Work	Technique	Relevance to this project
Han & Orshansky [2]	Approximate computing paradigm	Motivation for trading accuracy for cost
Mittal [3]	Survey of techniques	Application domains, levels of approximation
Jiang <i>et al.</i> [4]	Survey of approximate arithmetic	Error metrics used in Chapter 6
Kulkarni <i>et al.</i> [5]	Under-designed 2×2 block	Approximate building block
Rehman <i>et al.</i> [6]	Architectural space exploration	Placement of approximate blocks
Ullah <i>et al.</i> [7]	FPGA-specific multipliers	FPGA cost differs from ASIC cost
Vakili <i>et al.</i> [8]	Dynamic LUT reconfiguration	State-of-the-art LUT savings
Kida & Sato [9]	Hand-structured 4-bit multiplier	Structure matters on 7-series LUTs

Chapter 3

Theoretical Background

3.1 Binary Multiplication

For unsigned n -bit operands $A = \sum_{i=0}^{n-1} a_i 2^i$ and $B = \sum_{j=0}^{n-1} b_j 2^j$, the product is

$$P = AB = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_i b_j 2^{i+j}. \quad (3.1)$$

Each term $a_i b_j$ is a partial-product bit produced by an AND gate. An array multiplier adds these n^2 bits with rows of adders. The product of two 8-bit numbers needs 16 bits, and the largest product is $255 \times 255 = 65,025$.

3.2 Recursive (Divide-and-Conquer) Multiplication

Splitting each operand into a high half and a low half, $A = 2^{n/2} A_H + A_L$ and $B = 2^{n/2} B_H + B_L$, gives

$$AB = 2^n A_H B_H + 2^{n/2} (A_H B_L + A_L B_H) + A_L B_L. \quad (3.2)$$

An 8×8 multiplier therefore needs four 4×4 multipliers and an adder, and each 4×4 multiplier needs four 2×2 multipliers. Equation (3.2) makes the *significance* of each partial product explicit: an error in $A_L B_L$ is multiplied by 2^0 , an error in $A_H B_L$ or $A_L B_H$ by 2^4 , and an error in $A_H B_H$ by 2^8 .

3.3 Exact and Approximate 2×2 Multipliers

Table 3.1 gives the truth table of the exact and approximate 2×2 multipliers. The exact block needs four output bits:

$$p_0 = a_0 b_0, \quad p_1 = a_1 b_0 \oplus a_0 b_1, \quad (3.3)$$

$$p_2 = a_1 b_1 \oplus a_0 b_1 b_0, \quad p_3 = a_1 a_0 b_1 b_0. \quad (3.4)$$

The under-designed block of Kulkarni *et al.* [5] replaces the XOR in p_1 with an OR and drops p_3 :

$$p_0 = a_0b_0, \quad p_1 = a_1b_0 \vee a_0b_1, \quad p_2 = a_1b_1, \quad p_3 = 0. \quad (3.5)$$

It differs from the exact block only for $A = B = 3$, where it returns 7 instead of 9 (an error of 2, i.e. 22%).

Table 3.1: Truth table of the exact and approximate 2×2 multipliers (products in decimal)

A	B	Exact	Approx	A	B	Exact	Approx
0	0	0	0	2	0	0	0
0	1	0	0	2	1	2	2
0	2	0	0	2	2	4	4
0	3	0	0	2	3	6	6
1	0	0	0	3	0	0	0
1	1	1	1	3	1	3	3
1	2	2	2	3	2	6	6
1	3	3	3	3	3	9	7

3.4 Partial-Product Truncation

Truncation removes a partial product completely. Removing $A_L B_L$ from (3.2) saves one 4×4 multiplier and one adder input. The maximum error is the largest value of $A_L B_L$, $15 \times 15 = 225$, which is 0.35% of the full-scale product. The error rate, however, is high, because $A_L B_L$ is non-zero for most inputs.

3.5 FPGA Architecture

3.5.1 Look-up tables and carry chains

Xilinx 7-series FPGAs are built from configurable logic blocks (CLBs), each containing two slices [10]. Each slice has four 6-input LUTs, eight flip-flops, wide-function multiplexers (MUXF7, MUXF8) and a 4-bit carry chain (CARRY4). A 6-input LUT can implement *any* Boolean function of up to six inputs, so the cost of a function depends on how many LUTs its outputs require, not on how many gates it contains.

3.5.2 Basys-3 board

The Digilent Basys-3 board [1] carries an Artix-7 XC7A35T-1CPG236C FPGA with 33,280 logic cells in 5,200 slices, 1,800 Kbit of block RAM and 90 DSP slices, together with 16 switches, 16 LEDs, five push-buttons, a four-digit seven-segment display and a 100 MHz oscillator. Designs are developed with the free Vivado WebPACK edition.

3.6 Error Metrics

Let P be the exact product, $\tilde{P}$ the approximate product, $ED = |P - \tilde{P}|$ the error distance, and $P_{\max} = 255^2$. Over all $N = 65,536$ input pairs [4]:

$$\text{ER} = \frac{\#\{ED > 0\}}{N} \times 100\%, \quad \text{MED} = \frac{1}{N} \sum ED, \quad (3.6)$$

$$\text{NMED} = \frac{\text{MED}}{P_{\max}} \times 100\%, \quad \text{MRED} = \frac{1}{N'} \sum_{P \neq 0} \frac{ED}{P} \times 100\%, \quad (3.7)$$

$$\text{WCE} = \max ED. \quad (3.8)$$

where N' is the number of input pairs with non-zero exact product.

3.7 Image Quality Metrics

For an $M \times N$ 8-bit image I and its approximate version $\tilde{I}$, the mean squared error and peak signal-to-noise ratio are

$$\text{MSE} = \frac{1}{MN} \sum_{x,y} (I(x,y) - \tilde{I}(x,y))^2, \quad \text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}} \text{ dB}. \quad (3.9)$$

The structural similarity index (SSIM) [11] compares local luminance, contrast and structure and ranges from 0 to 1, where 1 means identical images. PSNR above about 40 dB and SSIM above 0.99 are generally considered visually lossless.

Chapter 4

Proposed Design

4.1 Design Space

The 8×8 multiplier is built according to (3.2). For each of the four 4×4 partial products the designer can choose one of three options: exact (built from exact 2×2 blocks), approximate (built from under-designed 2×2 blocks), or removed (truncated). Following the significance of each term, approximation is introduced from the least-significant partial product upwards. Table 4.1 lists the six configurations implemented, and Figure 4.1 shows the proposed hybrid.

Table 4.1: Multiplier configurations (MODE parameter of mul8x8)

MODE	Name	$A_L B_L$ (2^0)	$A_H B_L$ (2^4)	$A_L B_H$ (2^4)	$A_H B_H$ (2^8)
0	EXACT	exact	exact	exact	exact
1	AM-L	approx	exact	exact	exact
2	AM-LM	approx	approx	approx	exact
3	AM-ALL	approx	approx	approx	approx
4	TRUNC-L	removed	exact	exact	exact
5	HYB	removed	approx	approx	exact

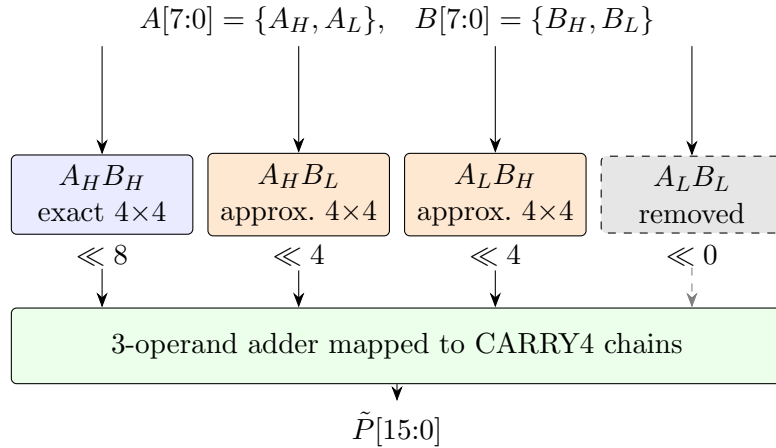


Figure 4.1: Architecture of the proposed hybrid (HYB) 8×8 multiplier. Each approximate 4×4 block is built from four under-designed 2×2 blocks.

4.2 Properties of the Design Family

- **One-sided error.** Both the under-designed block and truncation can only reduce a partial product, so every approximate design satisfies $\tilde{P} \leq P$. The error is always an under-estimate, which simplifies analysis and allows constant compensation.
- **Exact high-order term.** Except in AM-ALL, the most significant partial product $A_H B_H$ is exact, which bounds the error for large operands.
- **Bounded error for truncation.** TRUNC-L can never be wrong by more than 225.
- **Zero output for small operands.** TRUNC-L and HYB return zero whenever both operands are below 16, because the whole product lies in the removed $A_L B_L$ term. These designs therefore suit data dominated by mid-to-high values, such as natural images.

4.3 Verilog Implementation

4.3.1 2×2 blocks

Listing 4.1 shows the exact and approximate 2×2 blocks written directly from the Boolean equations of Section 3.3.

```

1  `timescale 1ns/1ps
2  // 2x2 unsigned multiplier building blocks.
3  //
4  // mul2x2_exact : accurate 2x2 multiplier, 4-bit product (3*3 = 9 = 4'b1001).
5  // mul2x2_approx : under-designed 2x2 multiplier of Kulkarni et al. (VLSI Design 2011).
6  // The only erroneous case is 3*3, which returns 7 (3'b111) instead of 9,
7  // so the product fits in 3 bits and the carry logic disappears.
8
9  module mul2x2_exact (
10     input wire [1:0] a,
11     input wire [1:0] b,
12     output wire [3:0] p
13 );
14     assign p[0] = a[0] & b[0];
15     assign p[1] = (a[1] & b[0]) ^ (a[0] & b[1]);
16     assign p[2] = (a[1] & b[1]) ^ (a[1] & b[0] & a[0] & b[1]);
17     assign p[3] = a[1] & b[1] & a[0] & b[0];
18 endmodule
19
20 module mul2x2_approx (
21     input wire [1:0] a,
22     input wire [1:0] b,
23     output wire [3:0] p
24 );

```

```

25   assign p[0] = a[0] & b[0];
26   assign p[1] = (a[1] & b[0]) | (a[0] & b[1]); // OR replaces XOR: 1+1 -> 1 (no carry)
27   assign p[2] = a[1] & b[1];
28   assign p[3] = 1'b0; // MSB never set
29 endmodule

```

Listing 4.1: 2×2 multiplier blocks (rtl/mul2x2.v)

4.3.2 4×4 block

The 4×4 module instantiates four 2×2 blocks inside a `generate` statement selected by the `APPROX` parameter, and combines them with shifts and additions (Listing 4.2).

```

1  'timescale 1ns/1ps
2  // 4x4 unsigned multiplier built from four 2x2 blocks.
3  //
4  // a = {aH, aL}, b = {bH, bL} (2-bit halves)
5  // p = (aH*bH << 4) + ((aH*bL + aL*bH) << 2) + aL*bL
6  //
7  // APPROX = 0 : all four 2x2 blocks are accurate -> exact 4x4 product
8  // APPROX = 1 : all four 2x2 blocks are approximate (Kulkarni-style)
9
10 module mul4x4 #(
11     parameter APPROX = 0
12 ) (
13     input wire [3:0] a,
14     input wire [3:0] b,
15     output wire [7:0] p
16 );
17     wire [3:0] pp_ll, pp_lh, pp_hl, pp_hh;
18
19     generate
20         if (APPROX) begin : g_approx
21             mul2x2_approx u_ll (.a(a[1:0]), .b(b[1:0]), .p(pp_ll));
22             mul2x2_approx u_lh (.a(a[1:0]), .b(b[3:2]), .p(pp_lh));
23             mul2x2_approx u_hl (.a(a[3:2]), .b(b[1:0]), .p(pp_hl));
24             mul2x2_approx u_hh (.a(a[3:2]), .b(b[3:2]), .p(pp_hh));
25         end else begin : g_exact
26             mul2x2_exact u_ll (.a(a[1:0]), .b(b[1:0]), .p(pp_ll));
27             mul2x2_exact u_lh (.a(a[1:0]), .b(b[3:2]), .p(pp_lh));
28             mul2x2_exact u_hl (.a(a[3:2]), .b(b[1:0]), .p(pp_hl));
29             mul2x2_exact u_hh (.a(a[3:2]), .b(b[3:2]), .p(pp_hh));
30         end
31     endgenerate
32
33     assign p = ({4'b0, pp_hh} << 4) + ({4'b0, pp_lh} << 2) + ({4'b0, pp_hl} << 2) + {4'b0, pp_ll};
34 endmodule

```

Listing 4.2: Recursive 4×4 multiplier (rtl/mul4x4.v)

4.3.3 8×8 multiplier

The 8×8 module (Listing 4.3) derives the configuration of each 4×4 block from the `MODE` parameter with `localparam` expressions, so a single module describes all six

designs. The file also contains the behavioural reference `mul8x8_beh`.

```

1  `timescale 1ns/1ps
2  // 8x8 unsigned multipliers.
3  //
4  // mul8x8_beh : behavioural reference, p = a * b (lets the synthesis tool choose).
5  // mul8x8 : recursive 8x8 multiplier built from four 4x4 blocks,
6  //
7  // a = {aH, aL}, b = {bH, bL} (4-bit halves)
8  // p = (aH*bH << 8) + ((aH*bL + aL*bH) << 4) + aL*bL
9  //
10 // The MODE parameter chooses where approximation is applied. Approximation is
11 // placed in the least-significant partial products first, because an error there
12 // is multiplied by a smaller weight (significance-driven placement).
13 //
14 // MODE 0 EXACT : all 4x4 blocks exact (accurate)
15 // MODE 1 AM-L : aL*bL approximate (lowest weight 2^0)
16 // MODE 2 AM-LM : aL*bL, aH*bL, aL*bH approximate; aH*bH exact
17 // MODE 3 AM-ALL : all four blocks approximate (fully under-designed multiplier)
18 // MODE 4 TRUNC-L : aL*bL block removed (forced to zero)
19 // MODE 5 HYB : aL*bL removed + aH*bL, aL*bH approximate; aH*bH exact (hybrid)
20
21 module mul8x8_beh (
22     input wire [7:0] a,
23     input wire [7:0] b,
24     output wire [15:0] p
25 );
26     assign p = a * b;
27 endmodule
28
29 module mul8x8 #(
30     parameter MODE = 0
31 ) (
32     input wire [7:0] a,
33     input wire [7:0] b,
34     output wire [15:0] p
35 );
36     localparam LL_APX = (MODE == 1) || (MODE == 2) || (MODE == 3);
37     localparam MID_APX = (MODE == 2) || (MODE == 3) || (MODE == 5);
38     localparam HH_APX = (MODE == 3);
39
40     wire [7:0] pp_ll, pp_lh, pp_hl, pp_hh;
41
42     generate
43         if (MODE == 4 || MODE == 5) begin : g_trunc
44             assign pp_ll = 8'd0;
45         end else begin : g_ll
46             mul4x4 #(.APPROX(LL_APX)) u_ll (.a(a[3:0]), .b(b[3:0]), .p(pp_ll));
47         end
48     endgenerate
49
50     mul4x4 #(.APPROX(MID_APX)) u_lh (.a(a[3:0]), .b(b[7:4]), .p(pp_lh));
51     mul4x4 #(.APPROX(MID_APX)) u_hl (.a(a[7:4]), .b(b[3:0]), .p(pp_hl));
52     mul4x4 #(.APPROX(HH_APX)) u_hh (.a(a[7:4]), .b(b[7:4]), .p(pp_hh));
53
54     assign p = ({8'b0, pp_hh} << 8) + ({8'b0, pp_lh} << 4) + ({8'b0, pp_hl} << 4) + {8'b0, pp_ll};
55 endmodule

```

Listing 4.3: Parameterised 8×8 multiplier (rtl/mul8x8.v)

4.3.4 Registered wrapper

For clocked operation, `mul8x8_pipe` registers the inputs and the output, giving a latency of two clock cycles (Appendix A).

4.4 Basys-3 Demonstration System

The top-level module `basys3_top` (Appendix C) instantiates all six multipliers and lets the user compare them on the board. Table 4.2 gives the user interface. Push-buttons are synchronised and sampled every 2^{20} clock cycles (about 10 ms) to remove contact bounce. The seven-segment display is multiplexed across its four digits.

Table 4.2: Basys-3 user interface

Board resource	Signal	Function
SW15–SW8	<code>a[7:0]</code>	Operand <i>A</i>
SW7–SW0	<code>b[7:0]</code>	Operand <i>B</i>
LD15–LD0	<code>led[15:0]</code>	Product of the selected multiplier (binary)
BTNU / BTND	<code>btnU</code> , <code>btnD</code>	Next / previous multiplier mode (0–5)
BTNC	<code>btnC</code>	Reset (mode 0)
7-segment digit 3	–	Current mode number
7-segment digits 2–0	–	Error $P - \tilde{P}$ in hexadecimal (low 12 bits)

Chapter 5

Implementation and Verification Methodology

5.1 Tools

- **Icarus Verilog** [12] (version 12) for RTL simulation and VCD waveform generation.
- **Yosys** [13] (version 0.33) for synthesis to Xilinx 7-series primitives using the `synth_xilinx` command with `-family xc7`.
- **Python 3** with NumPy, Matplotlib and scikit-image [14] for error analysis, image processing and plotting.
- **Xilinx Vivado** (optional) for place-and-route, timing, power and bitstream generation for the Basys-3 board, using the script in Appendix C.

5.2 Exhaustive Functional Verification

Because each operand has only 256 values, all $2^{16} = 65,536$ input pairs can be simulated. The testbench `tb_exhaustive` (Appendix B) instantiates the behavioural reference and all six designs, applies every input pair, and checks that:

1. the behavioural and recursive exact multipliers both equal $a \times b$;
2. no approximate design produces a product larger than $a \times b$.

It prints `RESULT: PASS` only if both checks hold for all inputs, and it writes every product to a text file for analysis. As an independent check, the Python analysis script contains a separate bit-accurate model of each design and confirms that it reproduces the RTL output for all 65,536 pairs.

5.3 Waveform Simulation

The clocked testbench `tb_wave` drives the registered wrappers of all six designs with a 100 MHz clock and a sequence of ten operand pairs that includes small values, the 3×3 corner case and the maximum 255×255 . It writes a VCD file, which the analysis script renders as a timing diagram.

5.4 Synthesis Methodology

Every design was synthesised for the Xilinx 7-series fabric with DSP inference disabled (`-nodsp`), so that all designs are compared on LUTs and carry chains. Two flows were used:

- **Hierarchical (primary).** Each module is technology-mapped separately and the netlist is flattened only for counting. The LUT count therefore reflects the structural design.
- **Flattened with ABC9 (sensitivity check).** The whole design is flattened before optimisation, allowing the tool to re-optimize across block boundaries.

Logic depth is the longest topological path through mapped cells (LUTs and CARRY4), excluding the input and output buffers, and serves as a technology-independent proxy for delay.

5.5 Image-Processing Applications

Three kernels were implemented in Python, with every multiplication looked up from the RTL product tables:

1. **Image multiplication:** $O = (I_1 \cdot I_2) \gg 8$ for the *camera* and *moon* test images (512×512).
2. **Gaussian smoothing:** 3×3 kernel with integer weights $\begin{bmatrix} 20 & 32 & 20 \\ 32 & 48 & 32 \\ 20 & 32 & 20 \end{bmatrix}$ (sum 256) applied to *camera* [15].
3. **Image blending:** $O = (\alpha I_1 + (255 - \alpha) I_2) \gg 8$ with $\alpha = 179$ (about 0.7), blending *camera* and *coins*.

Each approximate output is compared with the exact output using PSNR and SSIM.

Chapter 6

Results and Discussion

6.1 Functional Verification

The exhaustive testbench produced the following output:

```
Vectors applied : 65536
Exact-design mismatches : 0
Approx over-estimations : 0
RESULT: PASS
```

Listing 6.1: Exhaustive testbench console output

Both exact designs are therefore correct for every input, and every approximate design under-estimates or equals the exact product. The Python model matched the RTL output for all designs.

6.2 Simulation Waveforms

Figure 6.1 shows the clocked simulation. Each product appears two clock cycles after its operands because of the input and output registers. Table 6.1 lists the values.

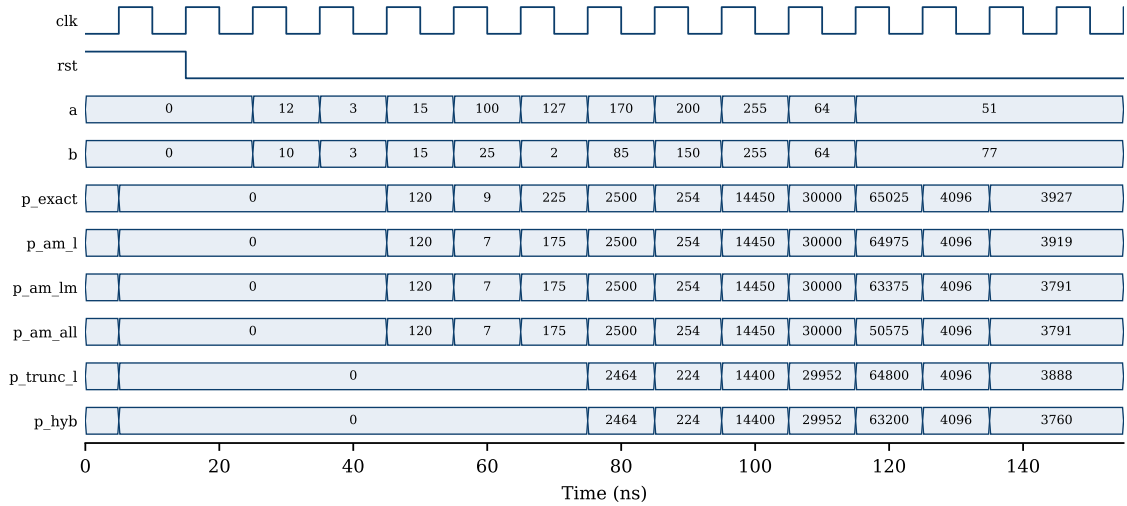


Figure 6.1: Simulation waveform of the six registered multipliers (Icarus Verilog, 100 MHz clock).

The 3×3 case shows the under-designed block returning 7. TRUNC-L and HYB return 0 for the first three pairs, whose upper nibbles are all zero, as predicted in Section 4.2. For 64×64 every design is exact, because all low-order fields are zero.

Table 6.1: Products observed in the waveform simulation

A	B	EXACT	AM-L	AM-LM	AM-ALL	TRUNC-L	HYB
12	10	120	120	120	120	0	0
3	3	9	7	7	7	0	0
15	15	225	175	175	175	0	0
100	25	2500	2500	2500	2500	2464	2464
127	2	254	254	254	254	224	224
170	85	14450	14450	14450	14450	14400	14400
200	150	30000	30000	30000	30000	29952	29952
255	255	65025	64975	63375	50575	64800	63200
64	64	4096	4096	4096	4096	4096	4096
51	77	3927	3919	3791	3791	3888	3760

6.3 Error Characteristics

Table 6.2 summarises the error metrics over all 65,536 input pairs, and Figures 6.2 and 6.3 show the distribution of the errors.

Table 6.2: Error metrics over all 65,536 input pairs

Design	ER (%)	MED	NMED (%)	MRED (%)	WCE	WCE (%)
EXACT	0.00	0.00	0.0000	0.000	0	0.00
AM-L	19.14	3.12	0.0048	0.076	50	0.08
AM-LM	40.67	103.12	0.1586	0.925	1650	2.54
AM-ALL	46.73	903.12	1.3889	3.280	14450	22.22
TRUNC-L	87.89	56.25	0.0865	1.859	225	0.35
HYB	90.28	156.25	0.2403	2.708	1825	2.81

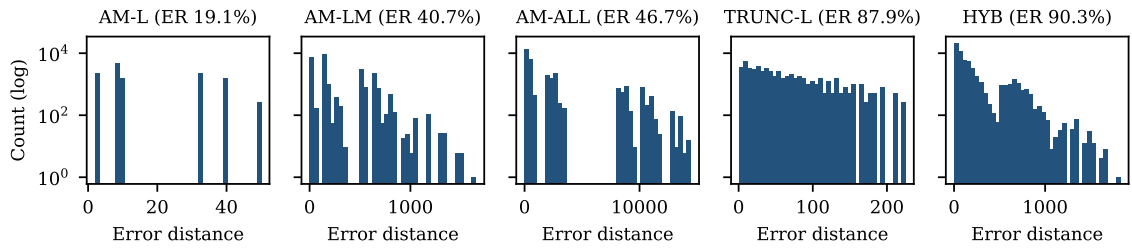


Figure 6.2: Histograms of non-zero error distances (logarithmic count axis).

Several observations follow:

- AM-L is extremely accurate (NMED 0.0048%, WCE 50), because its only errors are in the lowest-weight partial product.
- Extending the under-designed block to the middle partial products (AM-LM) raises NMED by a factor of 33, and extending it to $A_H B_H$ (AM-ALL) raises the

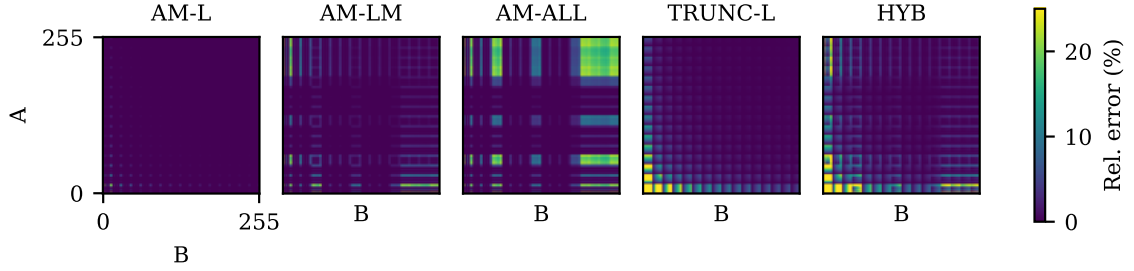


Figure 6.3: Relative error over the operand space (A vertical, B horizontal; colour scale clipped at 25%).

worst-case error to 22% of full scale.

- TRUNC-L is wrong for 88% of inputs, but its errors are small and bounded (WCE 225, NMED 0.0865%), so it is more accurate on average than AM-LM.
- HYB combines the two: NMED 0.24% and WCE 1825 (2.8% of full scale).
- For TRUNC-L and HYB, the largest relative errors appear at small operands, while AM-ALL also has large relative errors at large operands (Figure 6.3).

6.4 FPGA Resource Utilisation

Table 6.3 gives the synthesis results for the Xilinx 7-series fabric.

Table 6.3: Synthesis results for the Xilinx 7-series fabric (Yosys, DSP inference disabled)

Design	Hierarchical flow (primary)					Flat ABC9
	LUT	LUT6	CARRY4	Depth	Saving	LUT
Behavioural $a*b$	113 [†]	76	4	9	−0.9%	103
EXACT	112	19	12	8	—	103
AM-L	108	19	12	8	3.6%	120
AM-LM	100	19	12	8	10.7%	97
AM-ALL	96	19	12	8	14.3%	88
TRUNC-L	88	12	10	7	21.4%	82
HYB	80	12	10	7	28.6%	62

[†] plus 22 MUXF7 and 1 MUXF8 cells; with DSP inference allowed it maps to one DSP48E1.

6.4.1 Discussion

- The exact recursive design uses 112 LUTs, almost identical to the behavioural $a*b$ (113 LUTs plus 23 wide multiplexers). The recursive structure therefore costs nothing in the hierarchical flow.

- Each under-designed 2×2 block saves exactly one LUT, because its fourth output is the constant 0. AM-L, AM-LM and AM-ALL replace 4, 12 and 16 blocks and save 4, 12 and 16 LUTs respectively.
- Truncation removes a whole 4×4 block and one adder operand, saving 24 LUTs and two CARRY4 cells and shortening the logic depth from 8 to 7.
- HYB saves 32 LUTs (28.6%), the largest reduction among the designs.
- In the flattened flow the tool re-optimises across blocks. It confirms the large savings (HYB: 62 versus 103 LUTs, a 39.8% reduction), but AM-L becomes larger than EXACT (120 versus 103). Small approximations are therefore sensitive to synthesis heuristics, while the savings of truncation and HYB are robust.

6.5 Accuracy–Area Trade-off

Figure 6.4 plots LUT count against NMED. A design is *Pareto-optimal* if no other design is both smaller and more accurate. EXACT, AM-L, TRUNC-L and HYB are Pareto-optimal; AM-LM and AM-ALL are dominated, because TRUNC-L is both smaller and more accurate than either. The practical design rule is: *on LUT-based FPGAs, remove the lowest-weight partial product first, and approximate the middle partial products only if further savings are needed.*

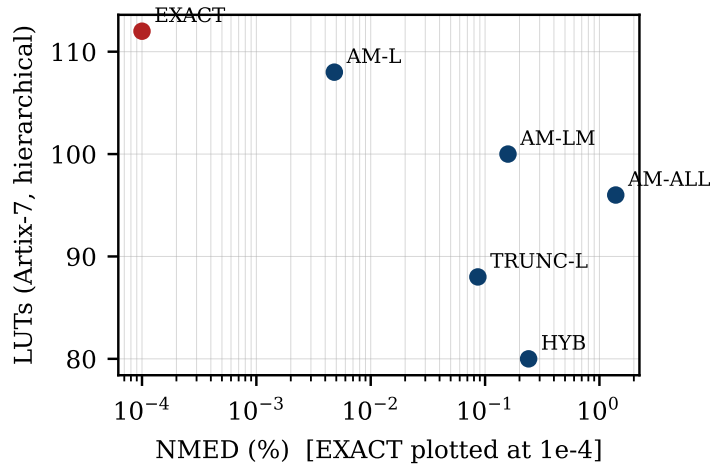


Figure 6.4: LUT count versus NMED (EXACT plotted at 10^{-4} for the logarithmic axis).

6.6 Image-Processing Results

Table 6.4 lists PSNR and SSIM for the three applications, and Figure 6.5 shows the output images.

Table 6.4: Image quality compared with exact multiplication: PSNR (dB) / SSIM

Design	Image multiplication	Gaussian smoothing	Image blending
AM-L	64.81 / 0.9998	identical (∞ / 1.0000)	60.93 / 0.9996
AM-LM	51.99 / 0.9973	56.17 / 0.9988	49.38 / 0.9980
AM-ALL	35.27 / 0.9851	34.98 / 0.9959	34.16 / 0.9951
TRUNC-L	55.70 / 0.9984	51.50 / 0.9976	51.99 / 0.9980
HYB	50.12 / 0.9960	49.71 / 0.9972	46.66 / 0.9975

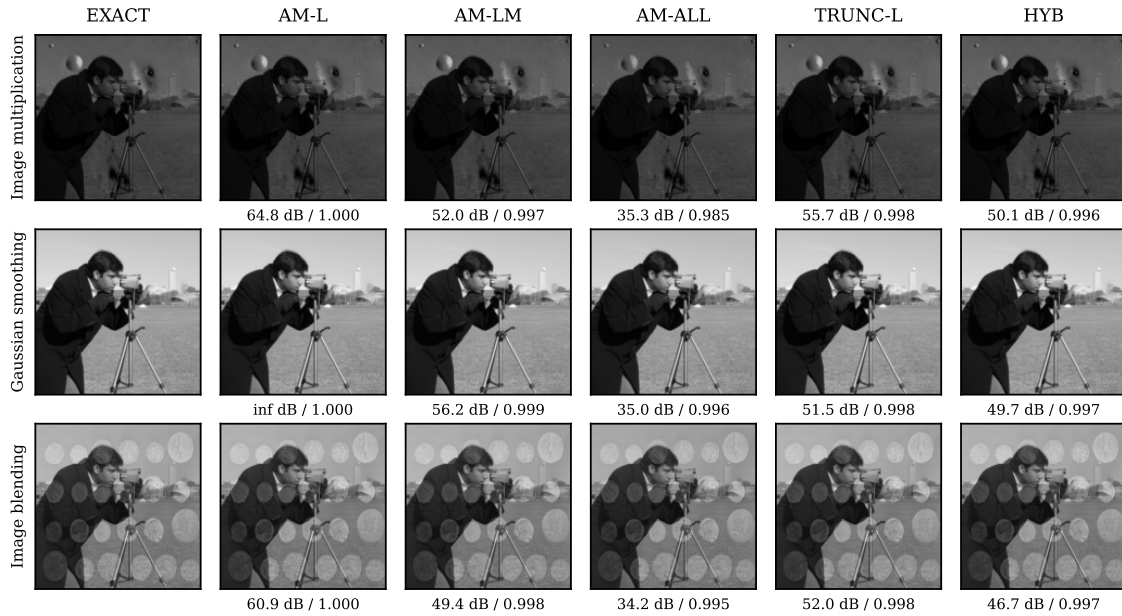


Figure 6.5: Output images of the three kernels for every multiplier (PSNR / SSIM against the exact output).

- Every design except AM-ALL achieves PSNR above 46 dB and SSIM of at least 0.996. At this level the outputs cannot be distinguished visually from the exact result (Figure 6.5).
- AM-L produces a bit-identical Gaussian-smoothing output. An AM-L error needs a 2-bit field equal to 3 in the low nibble of both operands, and the low nibbles of the kernel weights (20, 32, 48) never contain one.
- AM-ALL falls to about 34–35 dB, consistent with its much larger worst-case error.
- HYB keeps PSNR between 46.7 and 50.1 dB while using the fewest LUTs.

6.7 Basys-3 Demonstration Design

The complete demonstration system, with all six multipliers, the debounced mode selection and the seven-segment driver, synthesises to 657 LUTs, 78 CARRY4 cells and 75 flip-flops in the Yosys flow, about 3.2% of the 20,800 LUTs of the XC7A35T. Place-and-route, timing closure at 100 MHz, power estimation and bitstream generation are performed in Vivado with the script in Appendix C.

6.8 Limitations

- Resource figures come from an open-source synthesis flow. Vivado implementation results (post-route LUTs, maximum frequency and power) should be added from the reports generated by the provided script; absolute LUT counts will differ from Yosys.
- The error of all approximate designs is one-sided. A constant compensation term would reduce the mean error but has not been evaluated.
- TRUNC-L and HYB return zero when both operands are below 16, which makes them unsuitable for kernels that multiply many small numbers.
- The image applications were evaluated in software using the RTL product tables, not in a hardware image pipeline.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

A parameterised family of six recursive 8×8 unsigned multipliers was designed in Verilog HDL, verified exhaustively over all 65,536 input pairs, synthesised for the Xilinx 7-series (Artix-7) fabric used on the Basys-3 board, and evaluated in three image-processing applications. The main conclusions are:

1. The significance of the approximated partial product determines both the error and the saving. Approximating only $A_L B_L$ gives almost exact results (NMED 0.0048%), while approximating all four partial products gives a worst-case error of 22%.
2. On LUT-based FPGAs, removing the lowest partial product (TRUNC-L) saves more LUTs and gives lower mean error than applying the under-designed block to the middle and upper partial products.
3. The proposed hybrid design HYB saves 28.6% of LUTs in the hierarchical flow (39.8% in a flattened flow) with NMED 0.24%, and keeps image PSNR between 46.7 and 50.1 dB and SSIM of at least 0.996.

7.2 Future Scope

- Run the provided Vivado script to obtain post-route utilisation, timing and power on the Basys-3 board, and measure power on hardware.
- Add constant bias compensation to reduce the mean error of TRUNC-L and HYB.
- Extend the design to signed operands and to 16×16 multipliers.
- Integrate HYB into a streaming convolution or neural-network accelerator on the FPGA and evaluate end-to-end accuracy.
- Compare with Vivado's LogiCORE multiplier and with the DSP48E1 implementation for throughput and energy per operation.

Bibliography

- [1] *Basys 3 FPGA Board Reference Manual*, Digilent Inc.
- [2] J. Han and M. Orshansky, “Approximate computing: An emerging paradigm for energy-efficient design,” in *Proc. 18th IEEE European Test Symposium (ETS)*, 2013.
- [3] S. Mittal, “A survey of techniques for approximate computing,” *ACM Computing Surveys*, vol. 48, no. 4, pp. 62:1–62:33, 2016.
- [4] H. Jiang, F. J. H. Santiago, H. Mo, L. Liu, and J. Han, “Approximate arithmetic circuits: A survey, characterization, and recent applications,” *Proceedings of the IEEE*, vol. 108, no. 12, pp. 2108–2135, 2020.
- [5] P. Kulkarni, P. Gupta, and M. Ercegovac, “Trading accuracy for power with an underdesigned multiplier architecture,” in *Proc. 24th International Conference on VLSI Design*, 2011, pp. 346–351.
- [6] S. Rehman, W. El-Harouni, M. Shafique, A. Kumar, and J. Henkel, “Architectural-space exploration of approximate multipliers,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016.
- [7] S. Ullah, S. Rehman, M. Shafique, and A. Kumar, “High-performance accurate and approximate multipliers for FPGA-based hardware accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 2, pp. 211–224, 2022.
- [8] S. Vakili, M. Vaziri, A. Zarei, and J. M. P. Langlois, “DyRecMul: Fast and low-cost approximate multiplier for FPGAs using dynamic reconfiguration,” *ACM Transactions on Reconfigurable Technology and Systems*, 2024, doi: 10.1145/3663480; preprint arXiv:2310.10053.
- [9] Kida and Sato, “Hardware-efficient accurate 4-bit multiplier for Xilinx 7 series FPGAs,” arXiv:2510.21533, 2025.
- [10] *7 Series FPGAs Configurable Logic Block User Guide (UG474)*, Xilinx (AMD).
- [11] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

- [12] S. Williams, “Icarus Verilog,” <https://steveicarus.github.io/iverilog/>.
- [13] C. Wolf, “Yosys open SYnthesis suite,” <https://yosyshq.net/yosys/>.
- [14] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, “scikit-image: Image processing in Python,” *PeerJ*, vol. 2, p. e453, 2014.
- [15] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

Appendix A

Verilog RTL Source Code

The 2×2 , 4×4 and 8×8 modules are listed in Chapter 4 (Listings 4.1–4.3). The registered wrapper follows.

```
1  `timescale 1ns/1ps
2  // Registered wrapper used for the clocked waveform testbench and the FPGA top:
3  // inputs and outputs are registered, so the multiplier sits between two
4  // flip-flop stages (latency = 2 clock cycles).
5
6  module mul8x8_pipe #(
7      parameter MODE = 0
8  ) (
9      input wire clk,
10     input wire rst,
11     input wire [7:0] a,
12     input wire [7:0] b,
13     output reg [15:0] p
14 );
15     reg [7:0] a_r, b_r;
16     wire [15:0] p_w;
17
18     mul8x8 #(.MODE(MODE)) u_mul (.a(a_r), .b(b_r), .p(p_w));
19
20     always @(posedge clk) begin
21         if (rst) begin
22             a_r <= 8'd0;
23             b_r <= 8'd0;
24             p <= 16'd0;
25         end else begin
26             a_r <= a;
27             b_r <= b;
28             p <= p_w;
29         end
30     end
31 endmodule
```

Listing A.1: Registered wrapper (rtl/mul8x8_pipe.v)

Appendix B

Testbenches

```
1 // Exhaustive self-checking testbench.
2 //
3 // Applies all 256 x 256 = 65,536 input pairs to every design variant and
4 // 1. checks that the behavioural and recursive exact multipliers equal a*b,
5 // 2. checks that every approximate variant never over-estimates (p_apx <= a*b),
6 // 3. writes every product to sim/products.txt for error / image analysis:
7 // a b p_beh p_exact p_am_l p_am_lm p_am_all p_trunc_l p_hyb
8
9 `timescale 1ns/1ps
10
11 module tb_exhaustive;
12     reg [7:0] a, b;
13     wire [15:0] p_beh, p_ex, p_l, p_lm, p_all, p_tr, p_hy;
14
15     mul8x8_beh u_beh (.a(a), .b(b), .p(p_beh));
16     mul8x8 #(.MODE(0)) u_ex (.a(a), .b(b), .p(p_ex));
17     mul8x8 #(.MODE(1)) u_l (.a(a), .b(b), .p(p_l));
18     mul8x8 #(.MODE(2)) u_lm (.a(a), .b(b), .p(p_lm));
19     mul8x8 #(.MODE(3)) u_all (.a(a), .b(b), .p(p_all));
20     mul8x8 #(.MODE(4)) u_tr (.a(a), .b(b), .p(p_tr));
21     mul8x8 #(.MODE(5)) u_hy (.a(a), .b(b), .p(p_hy));
22
23     integer i, j, fd;
24     integer err_exact, err_over, n;
25     reg [15:0] ref;
26
27     initial begin
28         fd = $fopen("sim/products.txt", "w");
29         err_exact = 0;
30         err_over = 0;
31         n = 0;
32         for (i = 0; i < 256; i = i + 1) begin
33             for (j = 0; j < 256; j = j + 1) begin
34                 a = i;
35                 b = j;
36                 #1;
37                 ref = i * j;
38                 if (p_beh !== ref || p_ex !== ref) begin
39                     err_exact = err_exact + 1;
40                     if (err_exact <= 5)
41                         $display("EXACT MISMATCH a=%0d b=%0d beh=%0d rec=%0d ref=%0d", i, j, p_beh, p_ex
42                             , ref);
43                 end
44                 if (p_l > ref || p_lm > ref || p_all > ref || p_tr > ref || p_hy > ref)
45                     err_over = err_over + 1;
46                 $fdisplay(fd, "%0d %0d %0d %0d %0d %0d %0d %0d", i, j, p_beh, p_ex, p_l, p_lm,
47                     p_all, p_tr, p_hy);
48                 n = n + 1;
49             end
50         end
51         $fclose(fd);
```

```

50     $display("Vectors applied : %0d", n);
51     $display("Exact-design mismatches : %0d", err_exact);
52     $display("Approx over-estimations : %0d", err_over);
53     if (err_exact == 0 && err_over == 0)
54         $display("RESULT: PASS");
55     else
56         $display("RESULT: FAIL");
57     $finish;
58 end
59 endmodule

```

Listing B.1: Exhaustive self-checking testbench (tb/tb_exhaustive.v)

```

1  // Clocked testbench that produces sim/wave.vcd for the waveform figure.
2  // The exact (MODE 0) and five approximate pipelines receive the same
3  // operand stream; the product appears two clock cycles after the operands.
4
5  `timescale 1ns/1ps
6
7  module tb_wave;
8      reg clk = 1'b0;
9      reg rst = 1'b1;
10     reg [7:0] a = 8'd0, b = 8'd0;
11     wire [15:0] p_exact, p_am_l, p_am_lm, p_am_all, p_trunc_l, p_hyb;
12
13     always #5 clk = ~clk; // 100 MHz clock (Basys-3 oscillator frequency)
14
15     mul8x8_pipe #(.MODE(0)) u0 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_exact));
16     mul8x8_pipe #(.MODE(1)) u1 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_am_l));
17     mul8x8_pipe #(.MODE(2)) u2 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_am_lm));
18     mul8x8_pipe #(.MODE(3)) u3 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_am_all));
19     mul8x8_pipe #(.MODE(4)) u4 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_trunc_l));
20     mul8x8_pipe #(.MODE(5)) u5 (.clk(clk), .rst(rst), .a(a), .b(b), .p(p_hyb));
21
22     // Operand stream: small values, the 3x3 corner case, and large values.
23     reg [7:0] va [0:9];
24     reg [7:0] vb [0:9];
25     integer k;
26
27     initial begin
28         va[0] = 8'd12; vb[0] = 8'd10;
29         va[1] = 8'd3; vb[1] = 8'd3;
30         va[2] = 8'd15; vb[2] = 8'd15;
31         va[3] = 8'd100; vb[3] = 8'd25;
32         va[4] = 8'd127; vb[4] = 8'd2;
33         va[5] = 8'd170; vb[5] = 8'd85;
34         va[6] = 8'd200; vb[6] = 8'd150;
35         va[7] = 8'd255; vb[7] = 8'd255;
36         va[8] = 8'd64; vb[8] = 8'd64;
37         va[9] = 8'd51; vb[9] = 8'd77;
38
39         $dumpfile("sim/wave.vcd");
40         $dumpvars(0, tb_wave);
41
42         repeat (2) @(posedge clk);
43         rst <= 1'b0;
44         for (k = 0; k < 10; k = k + 1) begin
45             @(posedge clk);
46             a <= va[k];

```

```
47         b <= vb[k];
48     end
49     repeat (4) @(posedge clk);
50     $finish;
51 end
52
53 // Console log of each result (printed when it leaves the pipeline).
54 always @(posedge clk)
55     if (!rst)
56         $display("t=%0d ns exact=%0d AM-L=%0d AM-LM=%0d AM-ALL=%0d TRUNC-L=%0d HYB=%0d",
57             $time, p_exact, p_am_l, p_am_lm, p_am_all, p_trunc_l, p_hyb);
58 endmodule
```

Listing B.2: Clocked waveform testbench (tb/tb_wave.v)

Appendix C

FPGA Implementation Files

```

1  `timescale 1ns/1ps
2  // Basys-3 demonstration top level.
3  //
4  // sw[15:8] operand A sw[7:0] operand B
5  // led[15:0] 16-bit product of the selected multiplier
6  // btnU/btnD select the multiplier mode (0 EXACT, 1 AM-L, 2 AM-LM, 3 AM-ALL, 4 TRUNC-L, 5 HYB)
7  // 7-segment display: digit 3 shows the mode number,
8  // digits 2..0 show |exact - approx| in hex (error magnitude, low 12 bits)
9  // btnC synchronous reset
10
11 module basys3_top (
12     input wire clk, // 100 MHz
13     input wire [15:0] sw,
14     input wire btnC,
15     input wire btnU,
16     input wire btnD,
17     output wire [15:0] led,
18     output reg [6:0] seg, // active-low segments {g,f,e,d,c,b,a}
19     output wire dp,
20     output reg [3:0] an // active-low digit enables
21 );
22     wire rst = btnC;
23
24     // ----- operand registers -----
25     reg [7:0] a_r, b_r;
26     always @(posedge clk) begin
27         a_r <= sw[15:8];
28         b_r <= sw[7:0];
29     end
30
31     // ----- all six multipliers -----
32     wire [15:0] p [0:5];
33     mul8x8 #(.MODE(0)) m0 (.a(a_r), .b(b_r), .p(p[0]));
34     mul8x8 #(.MODE(1)) m1 (.a(a_r), .b(b_r), .p(p[1]));
35     mul8x8 #(.MODE(2)) m2 (.a(a_r), .b(b_r), .p(p[2]));
36     mul8x8 #(.MODE(3)) m3 (.a(a_r), .b(b_r), .p(p[3]));
37     mul8x8 #(.MODE(4)) m4 (.a(a_r), .b(b_r), .p(p[4]));
38     mul8x8 #(.MODE(5)) m5 (.a(a_r), .b(b_r), .p(p[5]));
39
40     // ----- button debounce + mode select -----
41     reg [19:0] div = 20'd0; // ~10 ms tick at 100 MHz
42     always @(posedge clk) div <= div + 1'b1;
43     wire tick = (div == 20'd0);
44
45     reg [1:0] u_sync, d_sync;
46     reg u_prev, d_prev;
47     reg [2:0] mode;
48     always @(posedge clk) begin
49         u_sync <= {u_sync[0], btnU};
50         d_sync <= {d_sync[0], btnD};
51         if (rst) begin

```

```

52     mode <= 3'd0;
53     u_prev <= 1'b0;
54     d_prev <= 1'b0;
55     end else if (tick) begin
56         u_prev <= u_sync[1];
57         d_prev <= d_sync[1];
58         if (u_sync[1] && !u_prev) mode <= (mode == 3'd5) ? 3'd0 : mode + 1'b1;
59         if (d_sync[1] && !d_prev) mode <= (mode == 3'd0) ? 3'd5 : mode - 1'b1;
60     end
61 end
62
63 // ----- outputs -----
64 reg [15:0] prod_r, err_r;
65 always @(posedge clk) begin
66     prod_r <= p[mode];
67     err_r <= p[0] - p[mode]; // approximate products never exceed exact
68 end
69 assign led = prod_r;
70 assign dp = 1'b1;
71
72 // ----- 7-segment multiplexing -----
73 reg [1:0] digit;
74 always @(posedge clk) if (div[16:0] == 17'd0) digit <= digit + 1'b1;
75
76 reg [3:0] nib;
77 always @(*) begin
78     case (digit)
79         2'd3: begin an = 4'b0111; nib = {1'b0, mode}; end
80         2'd2: begin an = 4'b1011; nib = err_r[11:8]; end
81         2'd1: begin an = 4'b1101; nib = err_r[7:4]; end
82         default: begin an = 4'b1110; nib = err_r[3:0]; end
83     endcase
84     case (nib)
85         4'h0: seg = 7'b1000000; 4'h1: seg = 7'b1111001; 4'h2: seg = 7'b0100100; 4'h3: seg = 7'
            b0110000;
86         4'h4: seg = 7'b0011001; 4'h5: seg = 7'b0010010; 4'h6: seg = 7'b0000010; 4'h7: seg = 7'
            b1111000;
87         4'h8: seg = 7'b0000000; 4'h9: seg = 7'b0010000; 4'hA: seg = 7'b0001000; 4'hB: seg = 7'
            b0000011;
88         4'hC: seg = 7'b1000110; 4'hD: seg = 7'b0100001; 4'hE: seg = 7'b0000110; default: seg = 7'
            b0001110;
89     endcase
90 end
91 endmodule

```

Listing C.1: Basys-3 top level (fpga/basys3_top.v)

```

1  ## Basys-3 constraints for basys3_top.v
2  ## Pin assignments follow the Digilent Basys-3 master XDC; confirm them against
3  ## the master file for your board revision before generating a bitstream.
4
5  ## 100 MHz clock
6  set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports clk]
7  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]
8
9  ## Switches
10 set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports {sw[0]}]
11 set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports {sw[1]}]
12 set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports {sw[2]}]

```

```

13 set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
14 set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
15 set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
16 set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
17 set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
18 set_property -dict { PACKAGE_PIN V2 IOSTANDARD LVCMOS33 } [get_ports {sw[8]}]
19 set_property -dict { PACKAGE_PIN T3 IOSTANDARD LVCMOS33 } [get_ports {sw[9]}]
20 set_property -dict { PACKAGE_PIN T2 IOSTANDARD LVCMOS33 } [get_ports {sw[10]}]
21 set_property -dict { PACKAGE_PIN R3 IOSTANDARD LVCMOS33 } [get_ports {sw[11]}]
22 set_property -dict { PACKAGE_PIN W2 IOSTANDARD LVCMOS33 } [get_ports {sw[12]}]
23 set_property -dict { PACKAGE_PIN U1 IOSTANDARD LVCMOS33 } [get_ports {sw[13]}]
24 set_property -dict { PACKAGE_PIN T1 IOSTANDARD LVCMOS33 } [get_ports {sw[14]}]
25 set_property -dict { PACKAGE_PIN R2 IOSTANDARD LVCMOS33 } [get_ports {sw[15]}]
26
27 ## LEDs
28 set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports {led[0]}]
29 set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports {led[1]}]
30 set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports {led[2]}]
31 set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports {led[3]}]
32 set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports {led[4]}]
33 set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports {led[5]}]
34 set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports {led[6]}]
35 set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVCMOS33 } [get_ports {led[7]}]
36 set_property -dict { PACKAGE_PIN V13 IOSTANDARD LVCMOS33 } [get_ports {led[8]}]
37 set_property -dict { PACKAGE_PIN V3 IOSTANDARD LVCMOS33 } [get_ports {led[9]}]
38 set_property -dict { PACKAGE_PIN W3 IOSTANDARD LVCMOS33 } [get_ports {led[10]}]
39 set_property -dict { PACKAGE_PIN U3 IOSTANDARD LVCMOS33 } [get_ports {led[11]}]
40 set_property -dict { PACKAGE_PIN P3 IOSTANDARD LVCMOS33 } [get_ports {led[12]}]
41 set_property -dict { PACKAGE_PIN N3 IOSTANDARD LVCMOS33 } [get_ports {led[13]}]
42 set_property -dict { PACKAGE_PIN P1 IOSTANDARD LVCMOS33 } [get_ports {led[14]}]
43 set_property -dict { PACKAGE_PIN L1 IOSTANDARD LVCMOS33 } [get_ports {led[15]}]
44
45 ## 7-segment display
46 set_property -dict { PACKAGE_PIN W7 IOSTANDARD LVCMOS33 } [get_ports {seg[0]}]
47 set_property -dict { PACKAGE_PIN W6 IOSTANDARD LVCMOS33 } [get_ports {seg[1]}]
48 set_property -dict { PACKAGE_PIN U8 IOSTANDARD LVCMOS33 } [get_ports {seg[2]}]
49 set_property -dict { PACKAGE_PIN V8 IOSTANDARD LVCMOS33 } [get_ports {seg[3]}]
50 set_property -dict { PACKAGE_PIN U5 IOSTANDARD LVCMOS33 } [get_ports {seg[4]}]
51 set_property -dict { PACKAGE_PIN V5 IOSTANDARD LVCMOS33 } [get_ports {seg[5]}]
52 set_property -dict { PACKAGE_PIN U7 IOSTANDARD LVCMOS33 } [get_ports {seg[6]}]
53 set_property -dict { PACKAGE_PIN V7 IOSTANDARD LVCMOS33 } [get_ports {dp}]
54 set_property -dict { PACKAGE_PIN U2 IOSTANDARD LVCMOS33 } [get_ports {an[0]}]
55 set_property -dict { PACKAGE_PIN U4 IOSTANDARD LVCMOS33 } [get_ports {an[1]}]
56 set_property -dict { PACKAGE_PIN V4 IOSTANDARD LVCMOS33 } [get_ports {an[2]}]
57 set_property -dict { PACKAGE_PIN W4 IOSTANDARD LVCMOS33 } [get_ports {an[3]}]
58
59 ## Buttons
60 set_property -dict { PACKAGE_PIN U18 IOSTANDARD LVCMOS33 } [get_ports btnC]
61 set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports btnU]
62 set_property -dict { PACKAGE_PIN U17 IOSTANDARD LVCMOS33 } [get_ports btnD]
63
64 ## Configuration
65 set_property CONFIG_VOLTAGE 3.3 [current_design]
66 set_property CFGBVS VCC0 [current_design]

```

Listing C.2: Basys-3 constraints (fpga/basys3.xdc)

```

1 # Vivado batch flow for the Basys-3 demo and per-design reports.
2 #

```

```

3 # vivado -mode batch -source fpga/vivado_build.tcl
4 #
5 # Produces, in fpga/vivado_out/:
6 # basys3_top.bit bitstream for the Basys-3 board
7 # util_basys3_top.rpt, timing_*.rpt, power_*.rpt
8 # util_mul8x8_MODE<n>.rpt out-of-context utilisation for each multiplier
9 # timing_mul8x8_MODE<n>.rpt (registered wrapper, 100 MHz constraint)
10 # Add these Vivado numbers to the paper/thesis tables next to the Yosys results.
11
12 set part xc7a35tcpg236-1
13 set here [file dirname [file normalize [info script]]]
14 set root [file dirname $here]
15 set out $here/vivado_out
16 file mkdir $out
17
18 set rtl [list $root/rtl/mul2x2.v $root/rtl/mul4x4.v $root/rtl/mul8x8.v $root/rtl/mul8x8_pipe.v]
19
20 # ----- per-design out-of-context runs (registered wrapper) -----
21 foreach mode {0 1 2 3 4 5} {
22     read_verilog $rtl
23     synth_design -top mul8x8_pipe -part $part -mode out_of_context -generic MODE=$mode -
        flatten_hierarchy rebuilt
24     create_clock -name clk -period 10.0 [get_ports clk]
25     opt_design
26     place_design
27     route_design
28     report_utilization -file $out/util_mul8x8_MODE$mode.rpt
29     report_timing_summary -file $out/timing_mul8x8_MODE$mode.rpt
30     report_power -file $out/power_mul8x8_MODE$mode.rpt
31     close_design
32 }
33
34 # ----- full Basys-3 demo -----
35 read_verilog [concat $rtl [list $here/basys3_top.v]]
36 read_xdc $here/basys3.xdc
37 synth_design -top basys3_top -part $part
38 opt_design
39 place_design
40 route_design
41 report_utilization -file $out/util_basys3_top.rpt
42 report_timing_summary -file $out/timing_basys3_top.rpt
43 report_power -file $out/power_basys3_top.rpt
44 write_bitstream -force $out/basys3_top.bit

```

Listing C.3: Vivado batch script (fpga/vivado_build.tcl)

Steps to program the Basys-3 board in Vivado (GUI)

1. Create a new RTL project for part xc7a35tcpg236-1 (Basys-3).
2. Add rtl/mul2x2.v, rtl/mul4x4.v, rtl/mul8x8.v and fpga/basys3_top.v as design sources, and fpga/basys3.xdc as the constraint file. Check the pin assignments against the Digilent master XDC.

3. Set `basys3_top` as the top module, then run Synthesis, Implementation and Generate Bitstream.
4. Open the Hardware Manager, connect to the board over USB and program the device.
5. Set operands on the switches, read the product on the LEDs, and press BT-NU/BTND to change the multiplier mode. The seven-segment display shows the mode and the error.

Appendix D

Synthesis and Analysis Scripts

```
1  #!/usr/bin/env bash
2  # Synthesise every multiplier variant for the Xilinx 7-series fabric (Artix-7,
3  # the Basys-3 device family) with Yosys and save one report per design.
4  #
5  # Primary flow (reports/<NAME>.stat) : hierarchy-preserving synthesis. Each 2x2 /
6  # 4x4 block is technology-mapped on its own, so the LUT count reflects
7  # the structural design; the netlist is flattened only for counting.
8  # Secondary flow (reports/<NAME>_flat.stat) : fully flattened with ABC9, where the tool
9  # re-optimises across block boundaries (sensitivity check).
10 #
11 # -nodsp : map multipliers into LUTs + CARRY4 instead of a DSP48E1 slice.
12 # ltp : longest topological path through mapped cells (logic depth, delay proxy).
13 set -euo pipefail
14 cd "$(dirname "$0")/.."
15 mkdir -p synth/reports
16
17 RTL="rtl/mul2x2.v rtl/mul4x4.v rtl/mul8x8.v"
18
19 run() { # name top mode
20     local name=$1 top=$2 mode=$3
21     local chp=""
22     [ "$mode" != "-" ] && chp="chparam -set MODE $mode $top;"
23     yosys -q -p "read_verilog $RTL; $chp synth_xilinx -family xc7 -nodsp -top $top; flatten; \
24         tee -o synth/reports/$name.stat stat; tee -o synth/reports/$name.ltp ltp -noff" >/dev/
25         null
26     yosys -q -p "read_verilog $RTL; $chp synth_xilinx -family xc7 -nodsp -flatten -abc9 -top $top; \
27         tee -o synth/reports/${name}_flat.stat stat" >/dev/null
28     echo "synthesised $name"
29 }
30
31 run BEH mul8x8_beh -
32 run EXACT mul8x8 0
33 run AM_L mul8x8 1
34 run AM_LM mul8x8 2
35 run AM_ALL mul8x8 3
36 run TRUNC_L mul8x8 4
37 run HYB mul8x8 5
38
39 # Behavioural multiplier with DSP inference allowed (what Vivado does by default)
40 yosys -q -p "read_verilog $RTL; synth_xilinx -family xc7 -top mul8x8_beh; \
41     tee -o synth/reports/BEH_DSP.stat stat" >/dev/null
42 echo "synthesised BEH_DSP"
43
44 # FPGA demo top (checks that the Basys-3 design synthesises cleanly)
45 yosys -q -p "read_verilog $RTL fpga/basys3_top.v; synth_xilinx -family xc7 -nodsp -top basys3_top;
46     flatten; \
47     tee -o synth/reports/BASYS3_TOP.stat stat" >/dev/null
48 echo "synthesised BASYS3_TOP"
```

Listing D.1: Yosys synthesis script (synth/synth.sh)

```

1  #!/usr/bin/env bash
2  # Reproduce every result in the paper and thesis:
3  # 1. exhaustive RTL simulation (65,536 vectors x 7 designs, self-checking)
4  # 2. clocked waveform simulation (sim/wave.vcd)
5  # 3. Yosys synthesis for Xilinx 7-series (Artix-7 / Basys-3 family)
6  # 4. error metrics, image-quality metrics and all figures (results/)
7  # Requirements: iverilog, yosys, python3 with numpy, matplotlib, scikit-image.
8  set -euo pipefail
9  cd "$(dirname "$0")"
10 PY=${PYTHON:-python3}
11 mkdir -p sim results
12 RTL="rtl/mul2x2.v rtl/mul4x4.v rtl/mul8x8.v"
13
14 echo "== 1. exhaustive simulation"
15 iverilog -g2005 -o sim/tb_exhaustive.vvp tb/tb_exhaustive.v $RTL
16 vvp -n sim/tb_exhaustive.vvp | tee sim/tb_exhaustive.log
17 grep -q "RESULT: PASS" sim/tb_exhaustive.log
18
19 echo "== 2. waveform simulation"
20 iverilog -g2005 -o sim/tb_wave.vvp tb/tb_wave.v $RTL rtl/mul8x8_pipe.v
21 vvp -n sim/tb_wave.vvp | tee sim/tb_wave.log
22
23 echo "== 3. synthesis"
24 ./synth/synth.sh
25
26 echo "== 4. analysis and figures"
27 $PY analysis/analyze.py > results/analysis.log
28 cat results/error_metrics.csv results/synthesis.csv results/image_quality.csv

```

Listing D.2: One-command reproduction script (run_all.sh)

```

1  """Post-simulation analysis for the approximate 8x8 multiplier project.
2
3  Inputs (produced by the Verilog flow):
4  sim/products.txt exhaustive RTL simulation output (65,536 rows)
5  sim/wave.vcd clocked testbench waveform
6  synth/reports/*.stat/ltp Yosys synthesis reports
7
8  Outputs:
9  results/error_metrics.csv, results/synthesis.csv, results/image_quality.csv,
10 results/summary.json, results/fig_*.pdf / .png, results/img_*.png
11
12 All image-application results use the product tables produced by RTL simulation,
13 i.e. every multiplication in the image kernels is looked up from the Verilog output.
14 """
15
16 from __future__ import annotations
17
18 import csv
19 import json
20 import re
21 from pathlib import Path
22
23 import matplotlib
24
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27 import numpy as np

```

```

28 from skimage import data
29 from skimage.metrics import peak_signal_noise_ratio, structural_similarity
30 from skimage.transform import resize
31
32 ROOT = Path(__file__).resolve().parents[1]
33 RES = ROOT / "results"
34 RES.mkdir(exist_ok=True)
35
36 DESIGNS = ["EXACT", "AM_L", "AM_LM", "AM_ALL", "TRUNC_L", "HYB"]
37 LABEL = {"BEH": "Behavioural a*b", "EXACT": "EXACT", "AM_L": "AM-L", "AM_LM": "AM-LM",
38          "AM_ALL": "AM-ALL", "TRUNC_L": "TRUNC-L", "HYB": "HYB"}
39 COLS = {"EXACT": 3, "AM_L": 4, "AM_LM": 5, "AM_ALL": 6, "TRUNC_L": 7, "HYB": 8}
40 PMAX = 255 * 255
41
42 plt.rcParams.update({"font.family": "serif", "font.size": 8, "axes.titlesize": 8,
43                     "axes.labelsize": 8, "legend.fontsize": 7, "figure.dpi": 150})
44
45
46 # ----- products
47 def load_products() -> dict[str, np.ndarray]:
48     raw = np.loadtxt(ROOT / "sim/products.txt", dtype=np.int64)
49     assert raw.shape == (65536, 9), raw.shape
50     a, b = raw[:, 0], raw[:, 1]
51     assert np.array_equal(raw[:, 2], a * b), "behavioural model mismatch"
52     tables = {}
53     for d, c in COLS.items():
54         t = np.zeros((256, 256), dtype=np.int64)
55         t[a, b] = raw[:, c]
56         tables[d] = t
57     tables["REF"] = np.outer(np.arange(256), np.arange(256)).astype(np.int64)
58     return tables
59
60
61 def python_model(mode: int) -> np.ndarray:
62     """Independent bit-accurate Python model of mul8x8 (cross-checks the RTL)."""
63     def m2(a, b, apx):
64         if apx and a == 3 and b == 3:
65             return 7
66         return a * b
67
68     def m4(a, b, apx):
69         al, ah, bl, bh = a & 3, a >> 2, b & 3, b >> 2
70         return (m2(ah, bh, apx) << 4) + ((m2(ah, bl, apx) + m2(al, bh, apx)) << 2) + m2(al, bl, apx)
71
72     ll = mode in (1, 2, 3)
73     mid = mode in (2, 3, 5)
74     hh = mode == 3
75     t = np.zeros((256, 256), dtype=np.int64)
76     for a in range(256):
77         for b in range(256):
78             al, ah, bl, bh = a & 15, a >> 4, b & 15, b >> 4
79             pll = 0 if mode in (4, 5) else m4(al, bl, ll)
80             t[a, b] = (m4(ah, bh, hh) << 8) + ((m4(ah, bl, mid) + m4(al, bh, mid)) << 4) + pll
81     return t
82
83
84 def error_metrics(t: np.ndarray, ref: np.ndarray) -> dict:
85     ed = (ref - t).astype(np.float64) # approximations never over-estimate (checked in RTL)
86     nz = ref > 0

```

```

87     red = np.zeros_like(ed)
88     red[nz] = np.abs(ed[nz]) / ref[nz]
89     return {
90         "ER_%": 100.0 * np.count_nonzero(ed) / ed.size,
91         "MED": float(np.mean(np.abs(ed))),
92         "NMED_%": 100.0 * float(np.mean(np.abs(ed))) / PMAX,
93         "MRED_%": 100.0 * float(np.mean(red[nz])),
94         "WCE": int(np.max(np.abs(ed))),
95         "WCE_%": 100.0 * float(np.max(np.abs(ed))) / PMAX,
96         "bias": float(np.mean(-ed)),
97     }
98
99
100 # ----- synthesis
101 def parse_stat(path: Path) -> dict:
102     txt = path.read_text()
103     cells = {m.group(1): int(m.group(2)) for m in re.finditer(r"^\s+([A-Z] [A-Z0-9_+])\s+(\d+)\s*$",
104         txt, re.M)}
105     lut = sum(v for k, v in cells.items() if re.fullmatch(r"LUT[1-6]", k))
106     return {"LUT": lut, "CARRY4": cells.get("CARRY4", 0), "MUXF7": cells.get("MUXF7", 0),
107         "MUXF8": cells.get("MUXF8", 0), "DSP48E1": cells.get("DSP48E1", 0),
108         "LUT6": cells.get("LUT6", 0)}
109
110 def parse_depth(path: Path) -> int:
111     m = re.search(r"length=(\d+)", path.read_text())
112     return int(m.group(1)) - 2 if m else -1 # minus input + output buffer
113
114
115 # ----- images
116 def approx_mul(t: np.ndarray, x: np.ndarray, y: np.ndarray) -> np.ndarray:
117     return t[x.astype(np.int64), y.astype(np.int64)]
118
119
120 def image_multiply(t, i1, i2):
121     return (approx_mul(t, i1, i2) >> 8).astype(np.uint8) # (A*B)/256
122
123
124 GAUSS = np.array([[20, 32, 20], [32, 48, 32], [20, 32, 20]], dtype=np.int64) # sum = 256
125
126
127 def gaussian(t, img):
128     p = np.pad(img.astype(np.int64), 1, mode="edge")
129     h, w = img.shape
130     acc = np.zeros((h, w), dtype=np.int64)
131     for dy in range(3):
132         for dx in range(3):
133             acc += approx_mul(t, p[dy:dy + h, dx:dx + w], np.full((h, w), GAUSS[dy, dx]))
134     return np.clip(acc >> 8, 0, 255).astype(np.uint8)
135
136
137 def blend(t, i1, i2, alpha=179): # alpha/255 ~ 0.70
138     return ((approx_mul(t, i1, np.full(i1.shape, alpha)) + approx_mul(t, i2, np.full(i2.shape, 255 -
139         alpha))) >> 8).astype(np.uint8)
140
141 # ----- VCD
142 def parse_vcd(path: Path, names: list[str]):
143     ids, series, t = {}, {n: [] for n in names}, 0

```

```

144     scope = []
145     for line in path.read_text().splitlines():
146         s = line.split()
147         if not s:
148             continue
149         if s[0] == "$scope":
150             scope.append(s[2])
151         elif s[0] == "$upscope":
152             scope.pop()
153         elif s[0] == "$var" and len(scope) == 1 and s[4] in names:
154             ids[s[3]] = s[4]
155         elif s[0].startswith("#"):
156             t = int(s[0][1:])
157         elif s[0][0] == "b" and len(s) == 2 and s[1] in ids:
158             v = s[0][1:]
159             series[ids[s[1]]].append((t, int(v, 2) if set(v) <= {"0", "1"} else None))
160         elif s[0][0] in "01xz" and s[0][1:] in ids:
161             series[ids[s[0][1:]]].append((t, int(s[0][0]) if s[0][0] in "01" else None))
162     return series
163
164
165 def plot_waveform(path_out: Path):
166     sigs = ["clk", "rst", "a", "b", "p_exact", "p_am_l", "p_am_lm", "p_am_all", "p_trunc_l", "p_hyb"]
167     ser = parse_vcd(ROOT / "sim/wave.vcd", sigs)
168     tend = max(tt for v in ser.values() for tt, _ in v) + 1
169     fig, ax = plt.subplots(figsize=(7.16, 3.4))
170     for row, name in enumerate(sigs):
171         y0 = len(sigs) - 1 - row
172         ch = ser[name] + [(tend, None)]
173         if name in ("clk", "rst"):
174             xs, ys = [], []
175             for (t0, v0), (t1, _) in zip(ch[:-1], ch[1:]):
176                 xs += [t0, t1]
177                 ys += [y0 + 0.1 + 0.6 * (v0 or 0)] * 2
178             ax.plot(xs, ys, color="#0b3d6b", lw=0.8)
179         else:
180             for (t0, v0), (t1, _) in zip(ch[:-1], ch[1:]):
181                 if t1 <= t0:
182                     continue
183                 ax.fill([t0 + 150, t0, t0 + 150, t1 - 150, t1, t1 - 150],
184                        [y0 + 0.1, y0 + 0.4, y0 + 0.7, y0 + 0.7, y0 + 0.4, y0 + 0.1],
185                        facecolor="#e8eef5", edgecolor="#0b3d6b", lw=0.6)
186                 if v0 is not None and t1 - t0 >= 8000:
187                     ax.text((t0 + t1) / 2, y0 + 0.4, str(v0), ha="center", va="center", fontsize=5.6)
188     ax.set_yticks([len(sigs) - 1 - i + 0.4 for i in range(len(sigs))])
189     ax.set_yticklabels([s.replace("_", "\\_") if False else s for s in sigs], fontsize=6.5)
190     ax.set_xlim(0, tend)
191     ax.set_ylim(-0.1, len(sigs))
192     ax.set_xticks(range(0, tend + 1, 20000))
193     ax.set_xticklabels([f"{x // 1000}" for x in range(0, tend + 1, 20000)])
194     ax.set_xlabel("Time (ns)")
195     for spine in ("top", "right", "left"):
196         ax.spines[spine].set_visible(False)
197     ax.tick_params(axis="y", length=0)
198     fig.tight_layout()
199     fig.savefig(path_out.with_suffix(".pdf"))
200     fig.savefig(path_out.with_suffix(".png"), dpi=200)
201     plt.close(fig)
202

```

```

203
204 # ----- main
205 def main():
206     tables = load_products()
207     ref = tables["REF"]
208
209     # Cross-check RTL against the independent Python model
210     for mode, d in enumerate(DESIGNS):
211         assert np.array_equal(python_model(mode), tables[d]), f"Python model mismatch for {d}"
212     print("RTL products match the independent Python model for all 5 designs")
213
214     # Error metrics
215     em = {d: error_metrics(tables[d], ref) for d in DESIGNS}
216     with open(RES / "error_metrics.csv", "w", newline="") as f:
217         w = csv.writer(f)
218         w.writerow(["design", "ER_%", "MED", "NMED_%", "MRED_%", "WCE", "WCE_%", "bias"])
219         for d in DESIGNS:
220             m = em[d]
221             w.writerow([LABEL[d], f"{m['ER_%']:.2f}", f"{m['MED']:.2f}", f"{m['NMED_%']:.4f}",
222                         f"{m['MRED_%']:.3f}", m["WCE"], f"{m['WCE_%']:.2f}", f"{m['bias']:.2f}"])
223
224     # Synthesis
225     syn = {}
226     for d in ["BEH"] + DESIGNS:
227         s = parse_stat(ROOT / f"synth/reports/{d}.stat")
228         s["depth"] = parse_depth(ROOT / f"synth/reports/{d}.ltp")
229         s["LUT_flat"] = parse_stat(ROOT / f"synth/reports/{d}_flat.stat")["LUT"]
230         syn[d] = s
231     syn["BEH_DSP"] = parse_stat(ROOT / "synth/reports/BEH_DSP.stat")
232     syn["BASYS3_TOP"] = parse_stat(ROOT / "synth/reports/BASYS3_TOP.stat")
233     base = syn["EXACT"]["LUT"]
234     with open(RES / "synthesis.csv", "w", newline="") as f:
235         w = csv.writer(f)
236         w.writerow(["design", "LUT", "LUT6", "CARRY4", "MUXF7", "MUXF8", "logic_depth", "
237                     LUT_saving_vs_EXACT_%", "LUT_flat_abc9"])
238         for d in ["BEH"] + DESIGNS:
239             s = syn[d]
240             w.writerow([LABEL[d], s["LUT"], s["LUT6"], s["CARRY4"], s["MUXF7"], s["MUXF8"], s["depth"],
241                         f"{100 * (base - s['LUT']) / base:.1f}", s["LUT_flat"]])
242
243     # Images
244     cam = data.camera().astype(np.uint8)
245     moon = data.moon().astype(np.uint8)
246     coins = (resize(data.coins(), (512, 512), anti_aliasing=True, preserve_range=True)).astype(np.
247                 uint8)
248     apps = {
249         "Image multiplication": lambda t: image_multiply(t, cam, moon),
250         "Gaussian smoothing": lambda t: gaussian(t, cam),
251         "Image blending": lambda t: blend(t, cam, coins),
252     }
253     iq = {}
254     outs = {}
255     for app, fn in apps.items():
256         exact = fn(ref)
257         iq[app] = {}
258         outs[app] = {"EXACT": exact}
259         for d in DESIGNS[1:]:
260             o = fn(tables[d])
261             outs[app][d] = o

```

```

260     iq[app][d] = {
261         "PSNR": float(peak_signal_noise_ratio(exact, o, data_range=255)) if not np.array_equal(
                exact, o) else float("inf"),
262         "SSIM": float(structural_similarity(exact, o, data_range=255)),
263     }
264 with open(RES / "image_quality.csv", "w", newline="") as f:
265     w = csv.writer(f)
266     w.writerow(["application"] + [f"{LABEL[d]} PSNR (dB)" for d in DESIGNS[1:]] + [f"{LABEL[d]}
        SSIM" for d in DESIGNS[1:]]
267     for app in apps:
268         w.writerow([app] + [f"{iq[app][d]['PSNR']:.2f}" for d in DESIGNS[1:]] + [f"{iq[app][d]['
        SSIM']:.4f}" for d in DESIGNS[1:]]])
269
270 # ----- figures
271 # 1. Area vs accuracy trade-off
272 fig, ax = plt.subplots(figsize=(3.4, 2.3))
273 for d in DESIGNS:
274     x, y = em[d]["NMED_%"], syn[d]["LUT"]
275     ax.scatter(max(x, 1e-4), y, s=22, color="#0b3d6b" if d != "EXACT" else "#b22222", zorder=3)
276     ax.annotate(LABEL[d], (max(x, 1e-4), y), textcoords="offset points", xytext=(4, 3), fontsize
        =6.5)
277 ax.set_xscale("log")
278 ax.set_xlabel("NMED (%) [EXACT plotted at 1e-4]")
279 ax.set_ylabel("LUTs (Artix-7, hierarchical)")
280 ax.grid(True, which="both", lw=0.3, alpha=0.5)
281 fig.tight_layout()
282 fig.savefig(RES / "fig_tradeoff.pdf")
283 fig.savefig(RES / "fig_tradeoff.png", dpi=200)
284 plt.close(fig)
285
286 # 2. Error-distance histograms
287 fig, axs = plt.subplots(1, 5, figsize=(7.16, 1.7), sharey=True)
288 for ax, d in zip(axs, DESIGNS[1:]):
289     ed = (ref - tables[d]).ravel()
290     nzed = ed[ed > 0]
291     ax.hist(nzed, bins=40, color="#23527c")
292     ax.set_title(f"{LABEL[d]} (ER {em[d]['ER_%']:.1f}%)")
293     ax.set_xlabel("Error distance")
294     ax.set_yscale("log")
295 axs[0].set_ylabel("Count (log)")
296 fig.tight_layout()
297 fig.savefig(RES / "fig_error_hist.pdf")
298 fig.savefig(RES / "fig_error_hist.png", dpi=200)
299 plt.close(fig)
300
301 # 3. Error heat maps (relative error over the operand space)
302 fig, axs = plt.subplots(1, 5, figsize=(7.16, 1.75))
303 for ax, d in zip(axs, DESIGNS[1:]):
304     red = np.where(ref > 0, (ref - tables[d]) / np.maximum(ref, 1), 0) * 100
305     im = ax.imshow(red, origin="lower", cmap="viridis", vmin=0, vmax=25)
306     ax.set_title(LABEL[d])
307     ax.set_xlabel("B")
308     first = d == DESIGNS[1]
309     ax.set_xticks([0, 255] if first else [])
310     ax.set_yticks([0, 255] if first else [])
311     axs[0].set_ylabel("A")
312     fig.subplots_adjust(wspace=0.15)
313     fig.colorbar(im, ax=axs, shrink=0.8, label="Rel. error (%)")
314     fig.savefig(RES / "fig_error_map.pdf", bbox_inches="tight")

```

```

315 fig.savefig(RES / "fig_error_map.png", dpi=200, bbox_inches="tight")
316 plt.close(fig)
317
318 # 4. Image outputs grid
319 fig, axs = plt.subplots(3, 6, figsize=(7.16, 3.9))
320 for r, app in enumerate(apps):
321     for c, d in enumerate(DESIGNS):
322         ax = axs[r, c]
323         ax.imshow(outs[app][d], cmap="gray", vmin=0, vmax=255)
324         ax.set_xticks([])
325         ax.set_yticks([])
326         if r == 0:
327             ax.set_title(LABEL[d])
328         if c == 0:
329             ax.set_ylabel(app, fontsize=7)
330         if d != "EXACT":
331             q = iq[app][d]
332             ax.set_xlabel(f"{q['PSNR']:.1f} dB / {q['SSIM']:.3f}", fontsize=6.5)
333 fig.tight_layout()
334 fig.savefig(RES / "fig_images.pdf")
335 fig.savefig(RES / "fig_images.png", dpi=200)
336 plt.close(fig)
337 for app in apps:
338     for d in DESIGNS:
339         key = {"Image multiplication": "multiply", "Gaussian smoothing": "gaussian", "Image
340             blending": "blend"}[app]
341         plt.imsave(RES / f"img_{key}_{d}.png", outs[app][d], cmap="gray", vmin=0, vmax=255)
342
343 # 5. Waveform
344 plot_waveform(RES / "fig_waveform")
345
346 summary = {"error_metrics": em, "synthesis": syn, "image_quality": iq}
347 (RES / "summary.json").write_text(json.dumps(summary, indent=2, default=float))
348 print(json.dumps(summary, indent=1, default=float)[:4000])
349
350 if __name__ == "__main__":
351     main()

```

Listing D.3: Analysis script (analysis/analyze.py)